Simplified AI Algorithms — Python

A concise collection of fundamental classical search algorithms, Bayesian inference implementations, and Hidden Markov Models (HMM).

1 Breadth-First Search (BFS)

```python
def bfs(graph, start, goal):
    queue = [start]
    visited = [start]
    parent = {start: None}

    while queue:
        node = queue.pop(0)

        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]

        for child in graph[node]:
            if child not in visited:
                visited.append(child)
                parent[child] = node
                queue.append(child)

    return None


graph = {
    'A': ['B', 'C'], 'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'], 'D': ['B'],
    'E': ['B', 'H'], 'F': ['C'],
    'G': ['C', 'I'], 'H': ['E'], 'I': ['G']
}

path = bfs(graph, 'A', 'I')
print("BFS Path:", path)
print("Cost:", len(path) - 1 if path else "No path")
```

2 Depth-First Search (DFS)

```python
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = []

    while stack:
        node, path = stack.pop()

        if node == goal:
            return path

        if node not in visited:
            visited.append(node)

            for child in reversed(graph[node]):
                if child not in visited:
                    stack.append((child, path + [child]))

    return None


graph = {
    'A': ['B', 'C'], 'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'], 'D': ['B'],
    'E': ['B', 'H'], 'F': ['C'],
    'G': ['C', 'I'], 'H': ['E'], 'I': ['G']
}

path = dfs(graph, 'A', 'I')
print("DFS Path:", path)
print("Cost:", len(path) - 1 if path else "No path")
```

3 Uniform-Cost Search (UCS)

```python
def ucs(graph, start, goal):
    frontier = [(0, start, [start])]
    visited = []

    while frontier:
        frontier.sort()
        cost, node, path = frontier.pop(0)

        if node in visited:
            continue

        visited.append(node)

        if node == goal:
            return path, cost

        for child, edge in graph[node]:
            if child not in visited:
                frontier.append((cost + edge, child, path + [child]))

    return None, float('inf')


graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 2), ('G', 5)],
    'D': [], 'E': [('H', 1)],
    'F': [], 'G': [('I', 1)],
    'H': [], 'I': []
}

path, cost = ucs(graph, 'A', 'I')
print("UCS Path:", path)
print("Cost:", cost)
```

4 A* Search

```python
def astar(graph, start, goal, h):
    frontier = [(h[start], 0, start, [start])]
    visited = []

    while frontier:
        frontier.sort()
        f, cost, node, path = frontier.pop(0)

        if node in visited:
            continue

        visited.append(node)

        if node == goal:
            return path, cost

        for child, edge in graph[node]:
            if child not in visited:
                g = cost + edge
                frontier.append((g + h[child], g, child, path + [child]))

    return None, float('inf')


graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 2), ('G', 5)],
    'D': [], 'E': [('H', 1)],
    'F': [], 'G': [('I', 1)],
    'H': [], 'I': []
}

h = {'A': 7, 'B': 6, 'C': 4, 'D': 7,
     'E': 3, 'F': 3, 'G': 1, 'H': 2, 'I': 0}

path, cost = astar(graph, 'A', 'I', h)
print("A* Path:", path)
print("Cost:", cost)
```

5 Bayesian Network Enumeration

```python
def probability(value, parents, cpt, e):
    p = cpt[tuple(e[x] for x in parents)]
    return p if value else 1 - p


def calculate(vars, e, bn):
    if not vars:
        return 1

    v = vars[0]
    parents, cpt = bn[v]

    if v in e:
        return probability(e[v], parents, cpt, e) * calculate(vars[1:], e, bn)

    total = 0
    for value in [True, False]:
        new_e = e.copy()
        new_e[v] = value
        total += probability(value, parents, cpt, new_e) * calculate(vars[1:], new_e, bn)

    return total


def ask(x, e, bn):
    result = {}

    for value in [True, False]:
        new_e = e.copy()
        new_e[x] = value
        result[value] = calculate(list(bn), new_e, bn)

    total = sum(result.values())
    return {k: v / total for k, v in result.items()}


bn = {
    'B': ([], {(): 0.001}),
    'E': ([], {(): 0.002}),
    'A': (['B', 'E'], {
        (True, True): 0.95, (True, False): 0.94,
        (False, True): 0.29, (False, False): 0.001
    }),
    'J': (['A'], {(True,): 0.90, (False,): 0.05}),
    'M': (['A'], {(True,): 0.70, (False,): 0.01})
}

result = ask('B', {'J': True, 'M': True}, bn)
print("Bayesian Enumeration")
print("B=True:", round(result[True], 4))
print("B=False:", round(result[False], 4))
```

6 Short Bayesian Probability Calculation

Uses enumeration over hidden variables rather than formal variable elimination.

```python
from itertools import product

bn = {
    'R': ([], {(): 0.30}),
    'A': ([], {(): 0.05}),
    'T': (['R', 'A'], {
        (True, True): 0.90, (True, False): 0.70,
        (False, True): 0.60, (False, False): 0.10
    }),
    'D': (['T'], {(True,): 0.80, (False,): 0.20}),
    'M': (['T'], {(True,): 0.50, (False,): 0.05})
}


def prob(v, e):
    parents, cpt = bn[v]
    p = cpt[tuple(e[x] for x in parents)]
    return p if e[v] else 1 - p


result = {}

for r in [True, False]:
    total = 0

    for a, t in product([True, False], repeat=2):
        e = {'R': r, 'A': a, 'T': t, 'D': True, 'M': True}
        p = 1

        for v in bn:
            p *= prob(v, e)

        total += p

    result[r] = total

s = sum(result.values())

print("P(R | D=True, M=True)")
print("R=True:", round(result[True] / s, 4))
print("R=False:", round(result[False] / s, 4))
```

7 HMM: Filtering vs Fixed-Lag Smoothing

Implements forward filtering and backward fixed-lag smoothing over sequence observations.

```python
import numpy as np
import matplotlib.pyplot as plt

T = np.array([[0.7, 0.3],
              [0.3, 0.7]])

O = {1: [0.9, 0.2], 0: [0.1, 0.8]}

N, d = 20, 3
rng = np.random.default_rng(12)

hidden, observation = [], []
state = 0

for i in range(N):
    state = rng.choice(2, p=T[state])
    obs = int(rng.random() < O[1][state])
    hidden.append(state)
    observation.append(obs)

print("Observations:", observation)

F = np.array([0.5, 0.5])
filtered = []

for obs in observation:
    F = O[obs] * (T.T @ F)
    F = F / F.sum()
    filtered.append(F.copy())

filtered = np.array(filtered)

smoothed = []

for k in range(N - d):
    B = np.ones(2)

    for i in range(k + d, k, -1):
        B = T @ (O[observation[i]] * B)

    S = filtered[k] * B
    S = S / S.sum()
    smoothed.append(S)

smoothed = np.array(smoothed)

days = np.arange(1, N + 1)

plt.figure(figsize=(10, 5))
plt.plot(days, filtered[:, 0], "o-", label="Filtering: Rain probability")
plt.plot(days[:len(smoothed)], smoothed[:, 0], "s-",
         label="Fixed-Lag Smoothing")
plt.xlabel("Day")
plt.ylabel("Probability of Rain")
plt.title("HMM: Filtering vs Fixed-Lag Smoothing")
plt.ylim(0, 1)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Quick Revision Cheat Sheet

Algorithm	Core Concept / Key Structure
BFS	`Queue` — Level-by-level exploration
DFS	`Stack` — Go deep along branch first
UCS	`Priority Queue` — Lowest actual path cost $g(n)$
A* Search	`Priority Queue` — Actual cost + heuristic $f(n) = g(n) + h(n)$
Bayesian Enumeration	Enumerate all state possibilities and normalize
Short Bayesian Calculation	Multiply joint probabilities and sum hidden possibilities
HMM Filtering vs Smoothing	Forward vector update $P(X_t \| e_{1:t})$ vs d-step lookahead backward update $P(X_k \| e_{1:k+d})$